LLM: Foundations

- By Dr. Anand Jayaraman

© 2025 TekFrameworks

Role

Principal Scientist, TekFrameworks
https://www.linkedin.com/in/anandjayaraman1
Anand.Jayaraman@tekframeworks.com
Anandhan@iitbombay.org

Academics

Ph.D. in Physics, University of Pittsburgh, USA
B.Tech. IIT Bombay, India

Research interests

Financial Markets, Algorithmic Trading
Analytics in practice

© 2025 TekFrameworks

How do we understand language?

- By learning the meaning of a few words
  - Excellent is +ve; Devastated is –ve
- Sentiment analysis can be done by having a reference set of words with +ve and –ve connotations.
- We can reduce the number of words to store by transforming the words to its root representation (Devastation, Devastating, Devastated all have the same root)

© 2025 TekFrameworks

But, simple rules don’t always work

- The cost exceeded the project’s budget
- The product exceeded my expectations

- The Virus killed thousands
- The comedian absolutely killed it

- His services were cheap!
- The build quality is cheap.

© 2025 TekFrameworks

Can we represent words and its meaning with numbers?

	Noun/Adjective/ Verb/Connector	Sentiment?	Future?	Male?	…
Blue	2	-0.5	0	0.8	
Pink	2	0.8	0	-0.8	
cried	3	-1	-1	0	

© 2025 TekFrameworks

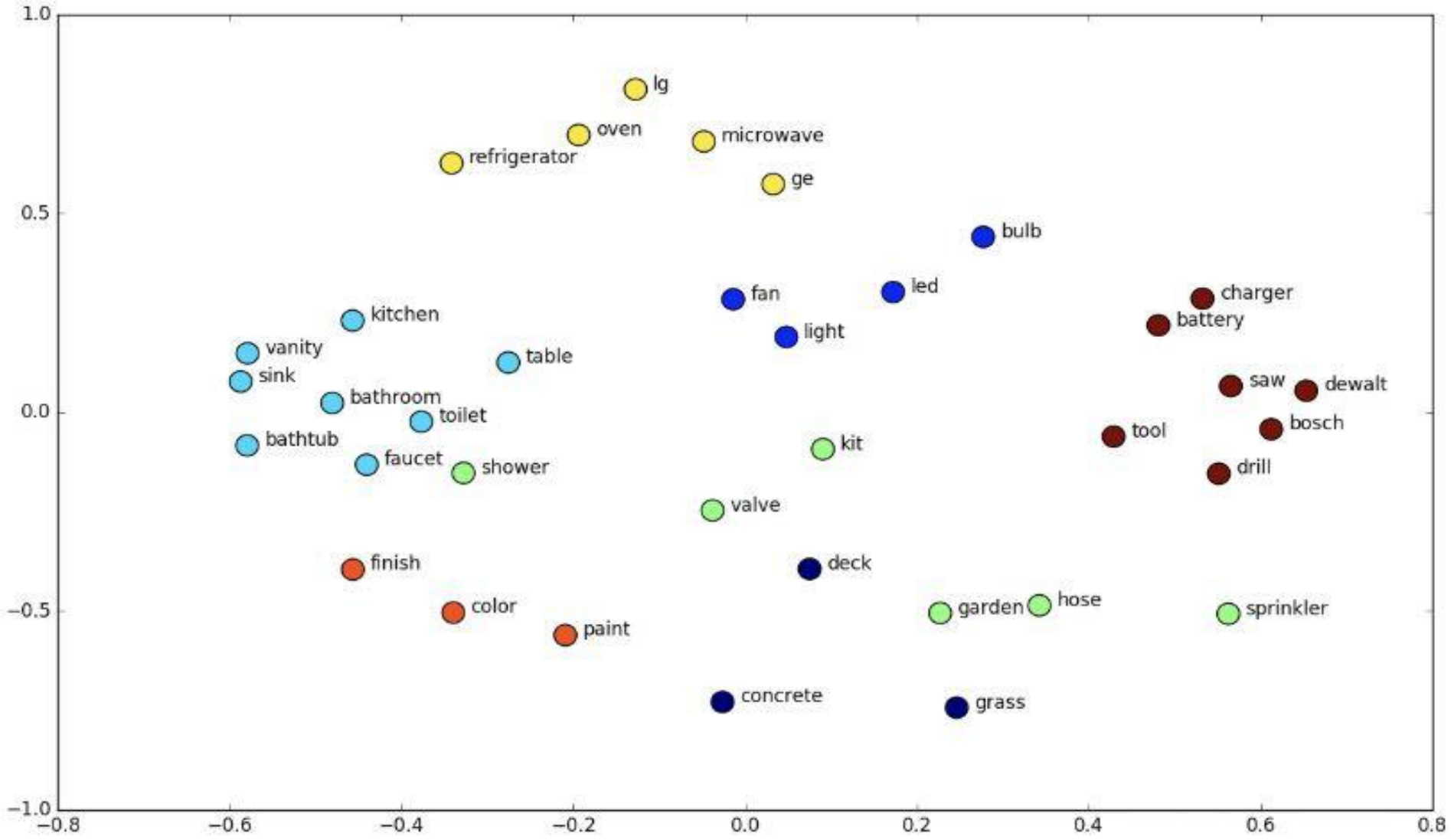
Word2Vec

- **Word2Vec** was one of the first breakthroughs that gave us such vector representations.

- In the embedding given by Word2Vec, words appearing in similar contexts have similar representations

© 2025 TekFrameworks

Word Embeddings



Source: https://web.engr.oregonstate.edu/

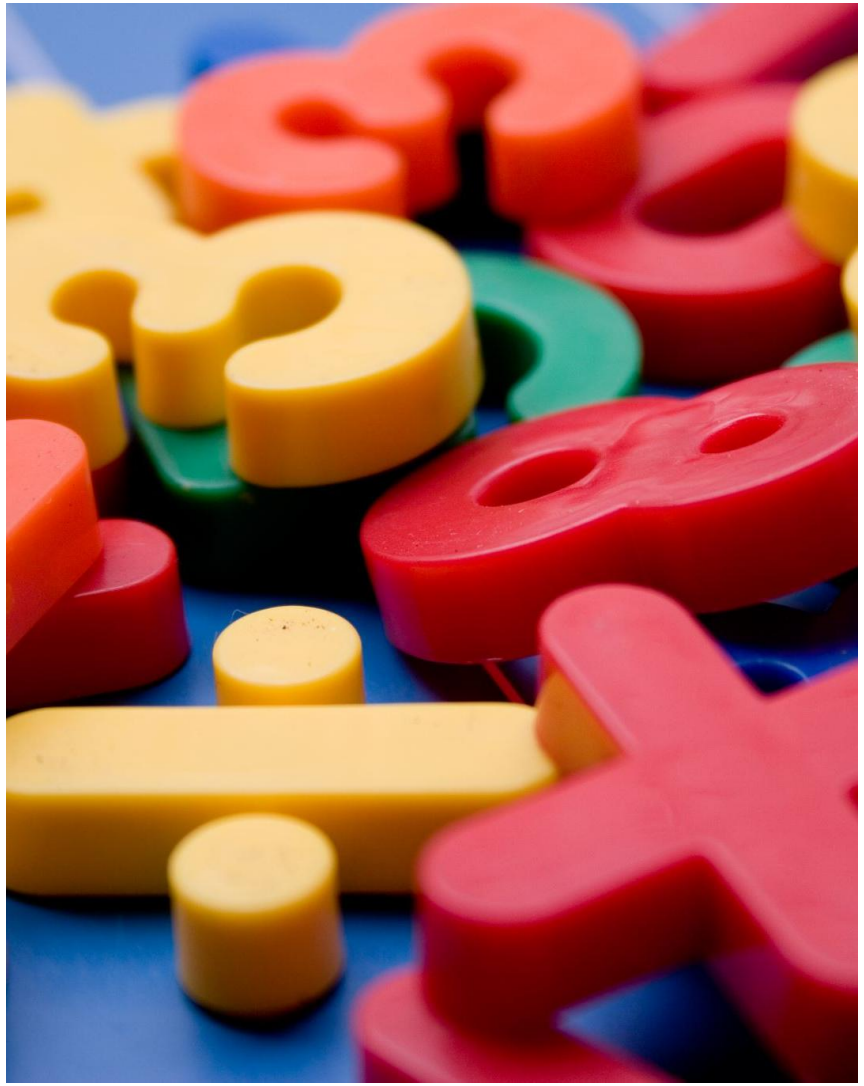
© 2025 TekFrameworks

How Vectors Power the World's Best Engines

It's not just words. It's "Behavior2Vec".

- Spotify:
  - Song2Vec treated songs as vectors. Two song vectors which are close are similar and hence can be recommended for the user.
  - **Result**: "Discover Weekly" engine that drives >40% of engagement.
- Netflix:
  - Movie2Vec codes users watch history as vectors. Vectors find hidden patterns (users who like “Dark Sci-fi” also like “Dystopian anime”)
  - **Result**: 80% of content watched on Netflix comes from recommendations
- Amazon:
  - Prod2Vec codes the purchase history as vectors. Finds relationship between unrelated items.
  - **Results**: 35% of total revenue comes from recommendation engine

© 2025 TekFrameworks

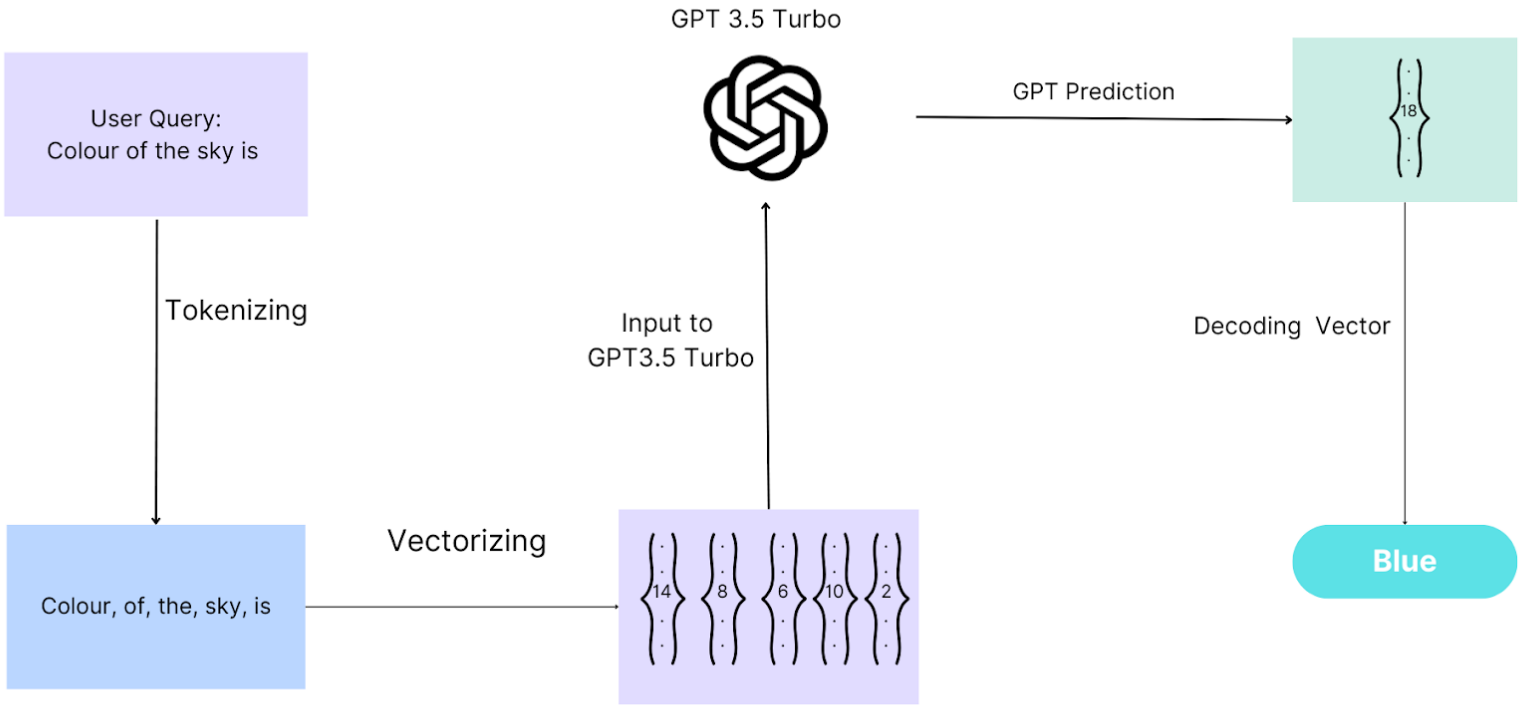


Generative Language Models

- The color of the sky is _____ ?
- For language models, predicting the above is equivalent to predicting the next number in the sequence 2, 4, 6, 8,____
- Probabilistic predictions

© 2025 TekFrameworks

What do LLM's do?



© 2025 TekFrameworks

Tokens : Pre-step to Embeddings

Natural Language → Tokens → Vectors

- LLMs don't "read words" — they process **tokens**.
- A *token* can be a:
  - **Word** (apple)
  - **Subword** (##ple)
  - **Character** (a, p, p, l, e)

- Tokenization is the **bridge** between human text and model input.

Different models use different tokenizers:
- **GPT** uses Byte Pair Encoding (BPE)
- **BERT** uses WordPiece

(Tokenizer - OpenAI API).

Text: In India, there is cricket and Bollywood.

↓

Tokenizer

↓

Tokens: ["in", "india", ",", "there", "is", "cricket", "and", "bolly", "##wood", "."]

© 2025 TekFrameworks

Embeddings: Limitation

- Word2vec was a good starting point. Converted words to rich numerical vectors
- However key limitation: Based on the context, meaning may change
  - I took my cell phone into the prison cell to meet the cell mates and collect their cell DNAs

© 2025 TekFrameworks

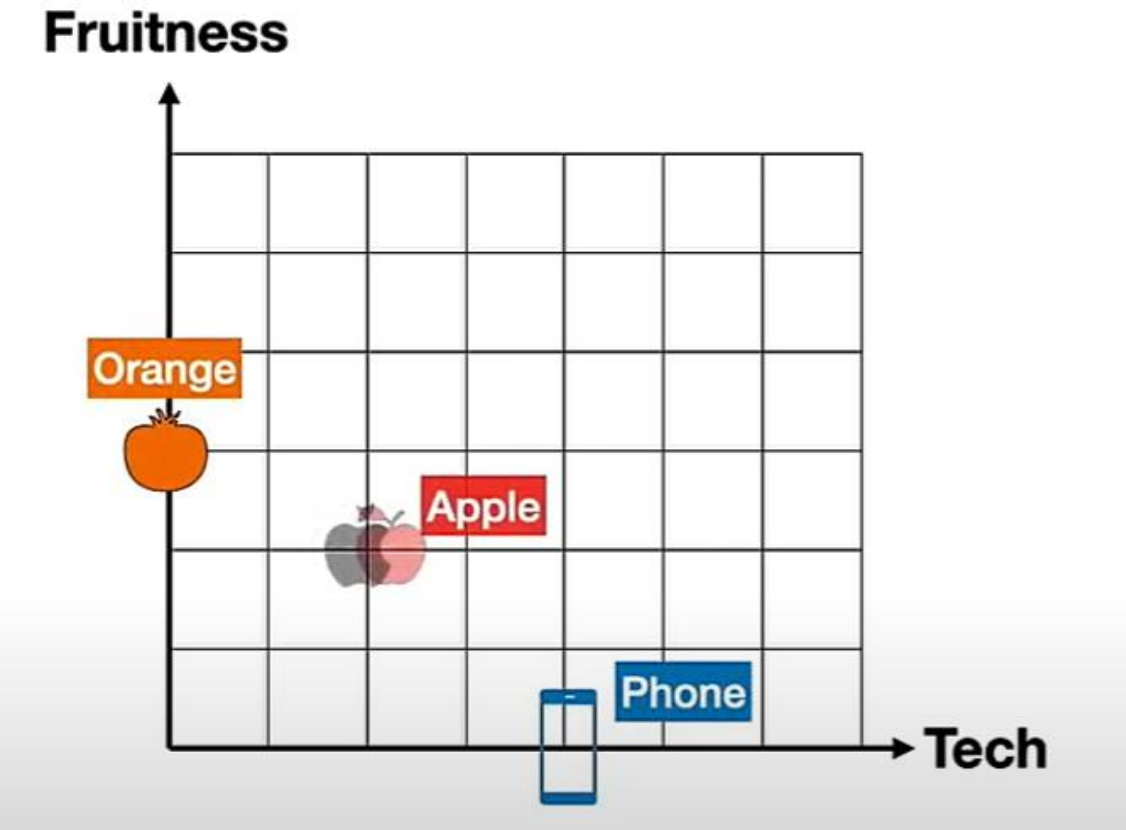
Attention block is the context finder

- A separate intervention called "***attention***" handles this problem elegantly. Attention weighs the importance of each word in predicting the next word

- The meaning of 'apple' is different in theses two sentences
  - I bought an apple and an orange

  - My apple phone is laggy

© 2025 TekFrameworks

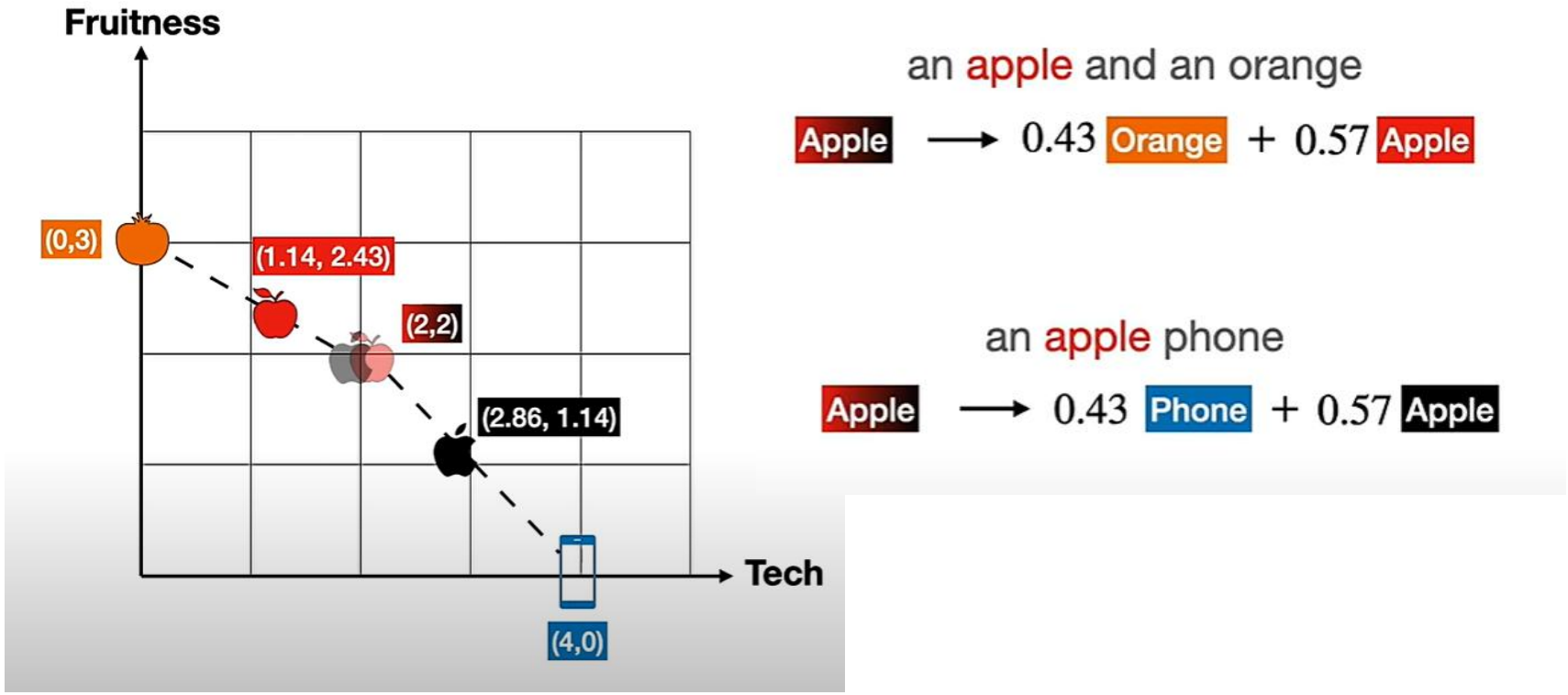


	Tech	Fruitness
Orange	0	3
Phone	4	0
Apple	2	2

From: https://www.youtube.com/watch?v=UPtG_38Oq8o

© 2025 TekFrameworks

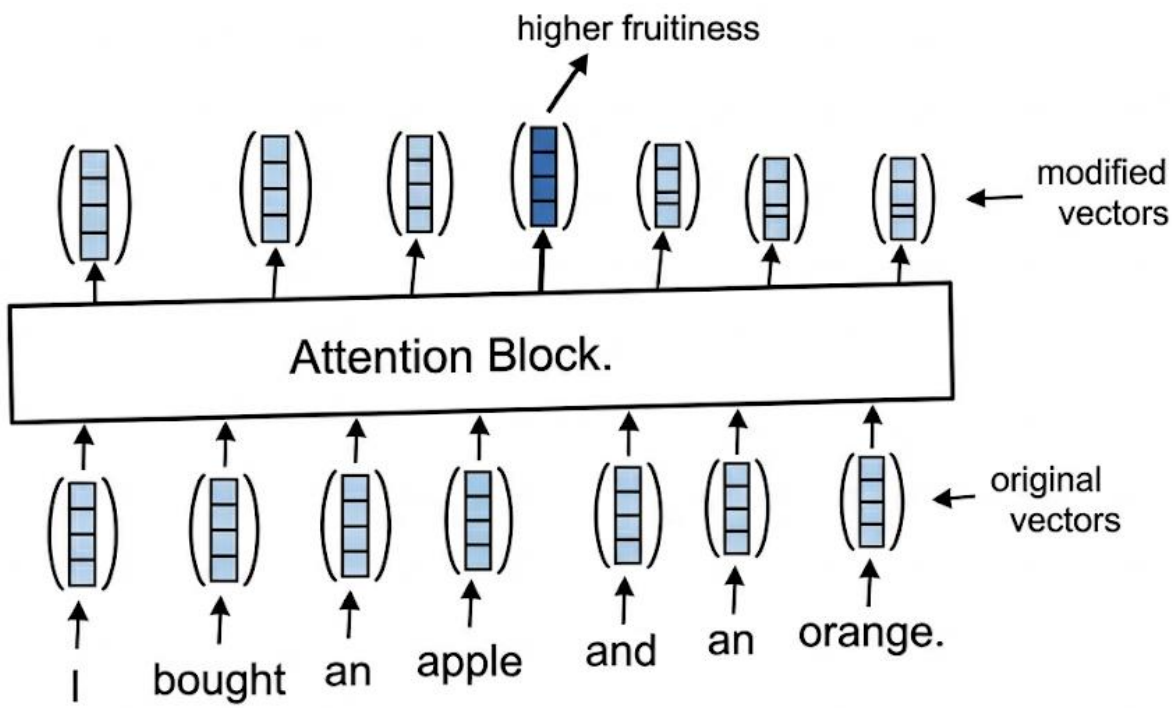
Attention block: Context finder



From: https://www.youtube.com/watch?v=UPtG_38Oq8o

© 2025 TekFrameworks

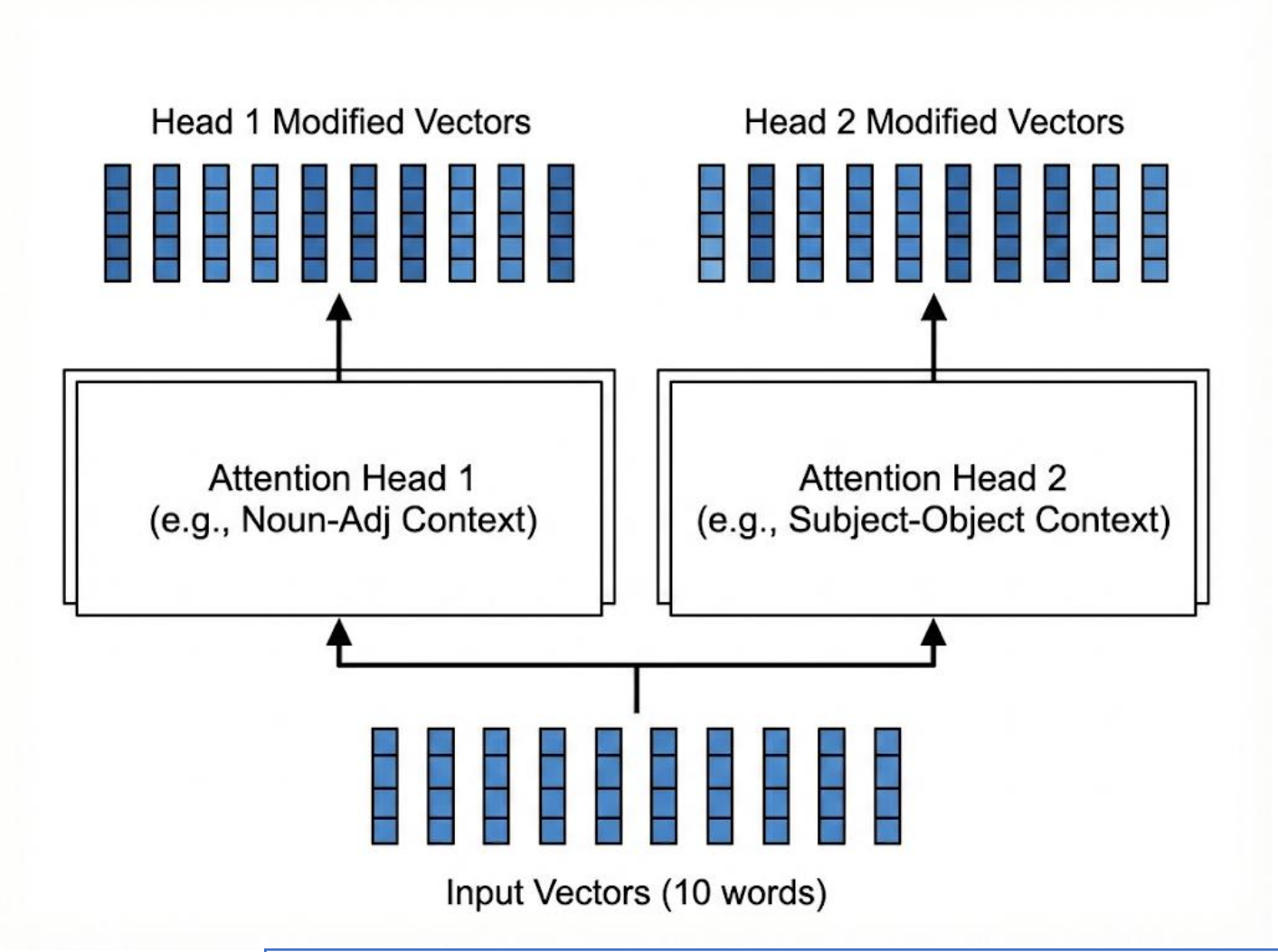
Attention Block



- The attention action is essentially accomplished through matrix multiplication operation.
- The numbers in the matrix are determined through training time – to accomplish the task of determining “fruitiness” or “techiness”
- Key point: Matrix operations can act on all words at one shot. So we have parallelization – the whole context being looked at in one shot.

© 2025 TekFrameworks

Multi-headed attention

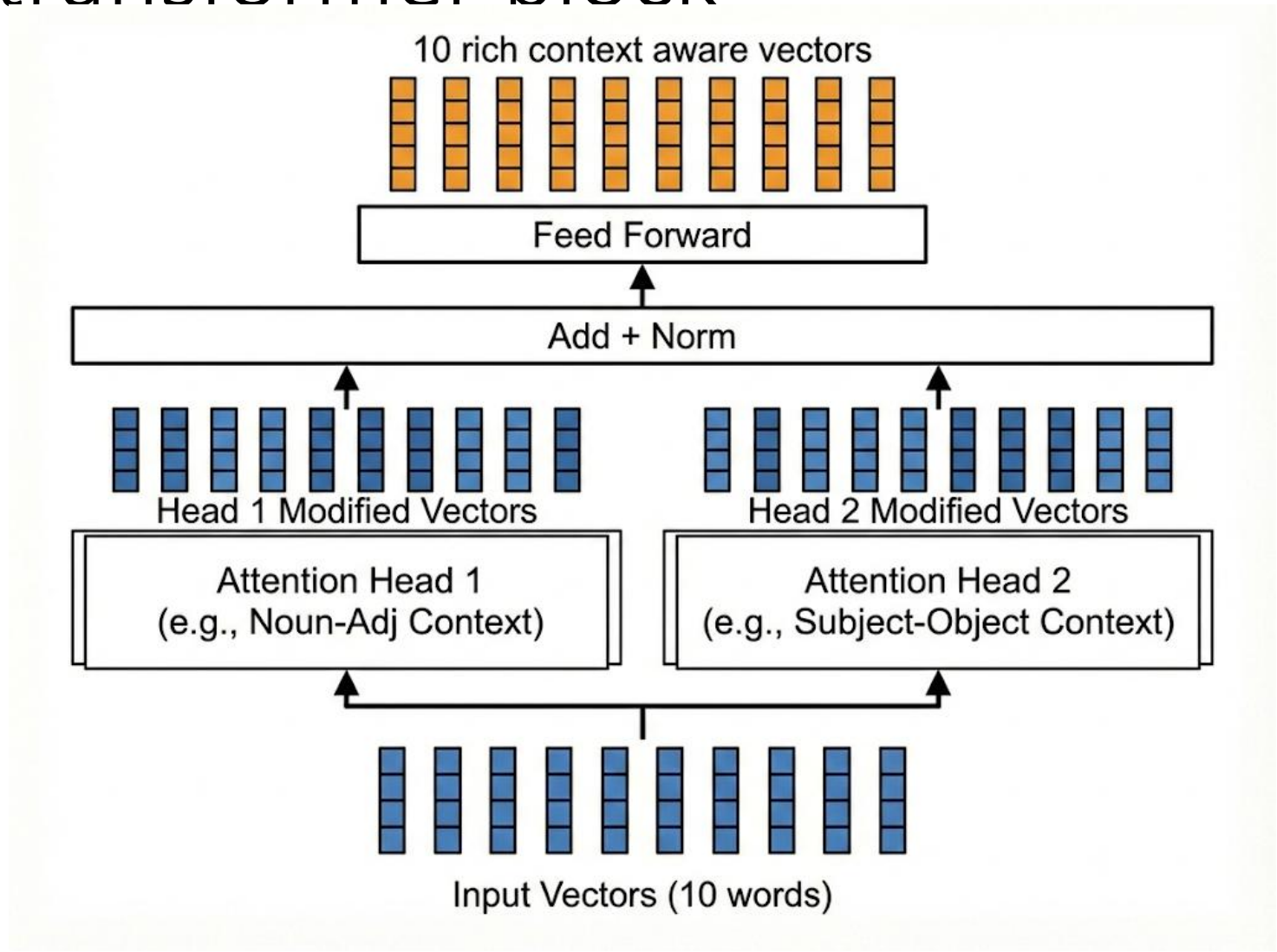


- Why is this useful?
- Imagine processing a legal contract.
  - Attention head-1 focusses on Timeline
  - Attention head-2 focusses on Liability
  - Attention head-3 focusses on Identity

Attention heads are like different “Departments” reviewing a document in parallel

© 2025 TekFrameworks

A basic transformer block



© 2025 TekFrameworks

Transformer Block

Step 1: Input Embedding

- Convert each token into a vector (word embedding)
- Add positional encoding to preserve word order

Step 2: Self-Attention Layer

- Each token attends to all other tokens in the sequence
- Helps model contextual relationships

Step 3: Add & Norm

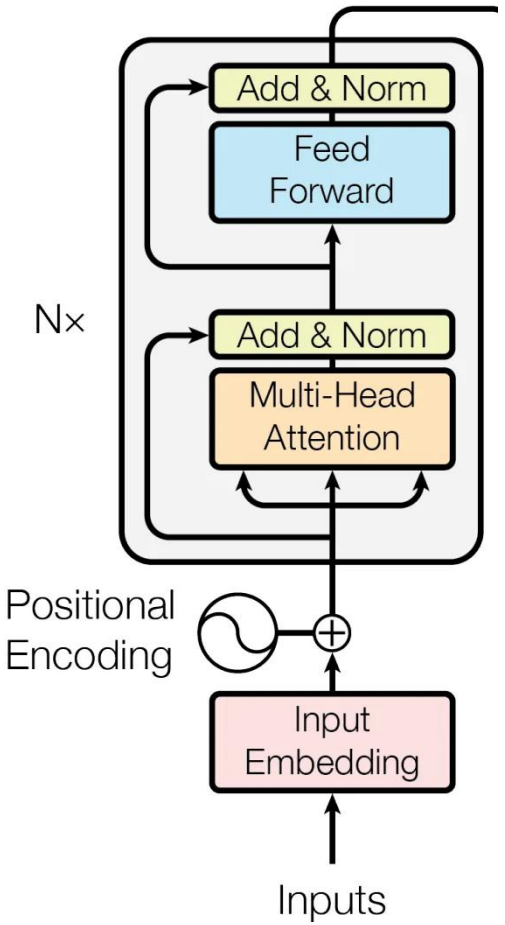
- Residual connection + Layer Normalization

Step 4: Feedforward Network

- Applies two linear layers with ReLU in between
- Transforms token-level representations

Step 5: Add & Norm (again)

- Another residual + LayerNorm applied

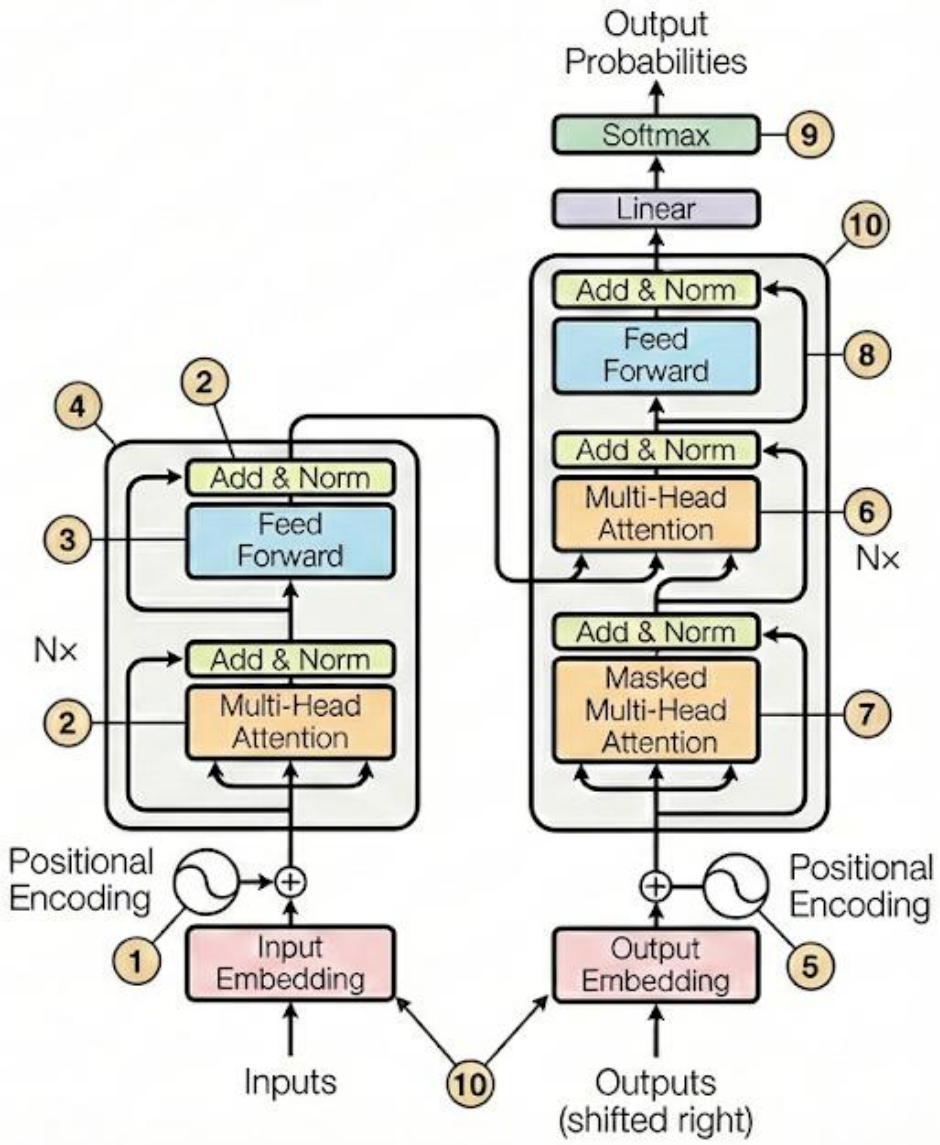


© 2025 TekFrameworks

Attention is all you need (Vaswani et al. 2017)

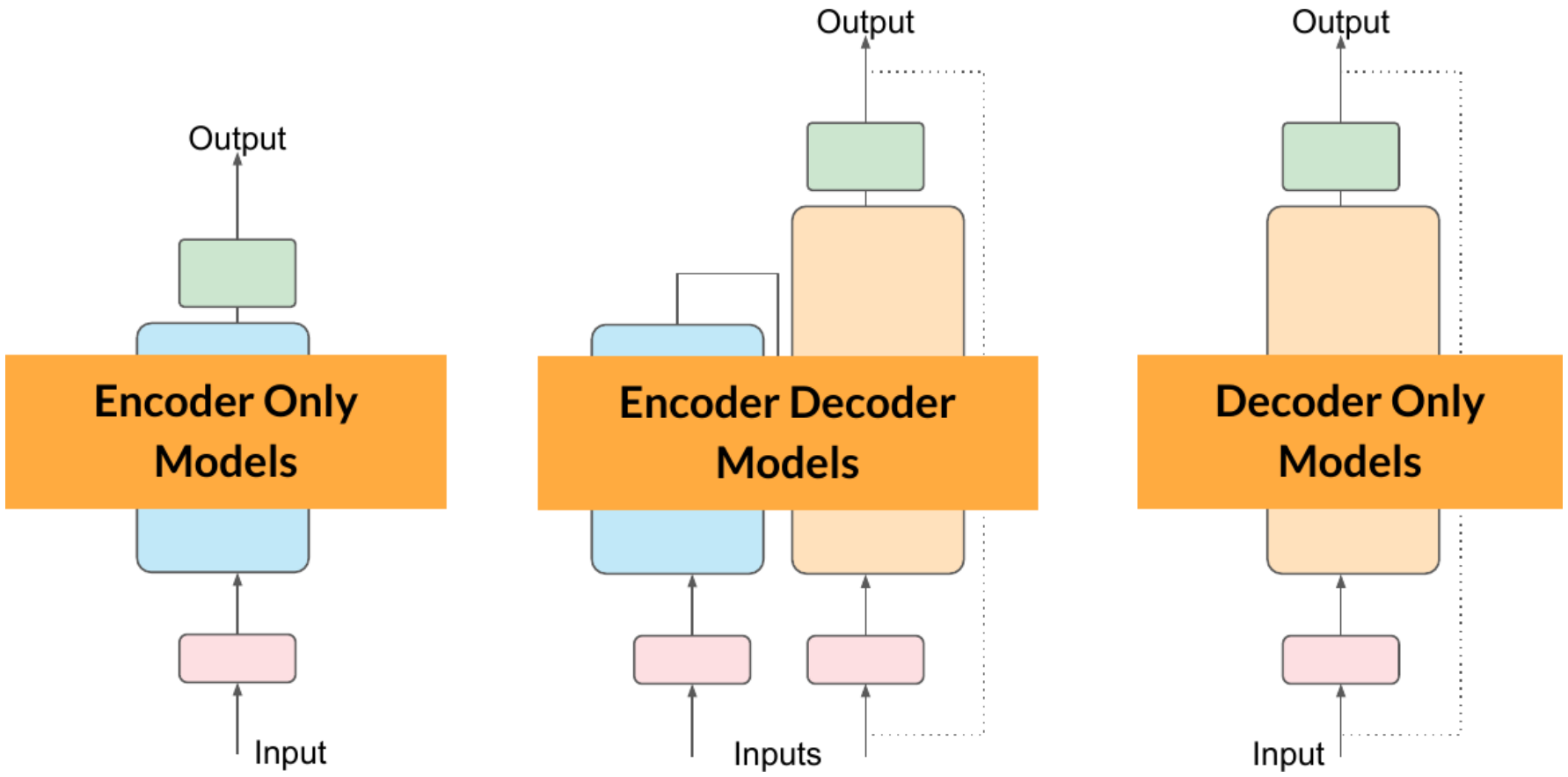
Transformer Architecture Annotations

1. Injects sequence order information, as the model has no inherencres of position).
2. Allows the model to jointly attend to (Multi-Head) from different representation subspaces at different positions.
3. Residual connections and Layer Normalization are crucial for training deep networks and preventing vanishing gradients.
4. Stacking N identical layers (e.g. (Encoder stack).
5. Similarly injects order for the target terquence (Decoder Positional Encoding).
6. Prevent positions from attending (e.g. (Masked Multi-Head Attention).
7. Performs attention over the output to focus (Decoder Multi-Head Attention).
8. Stacking N identical layers (e.g. (Decoder stack).
9. Converts the decoder's output (e.g.. (Softmax).
10. **Key Insight:** The Transformer relies entirely on attention mechanisms to draw global dependen-dencies between input and, dispensing.This allows for significant use and paralitalization and better handling of long-range dependencies.



© 2025 TekFrameworks

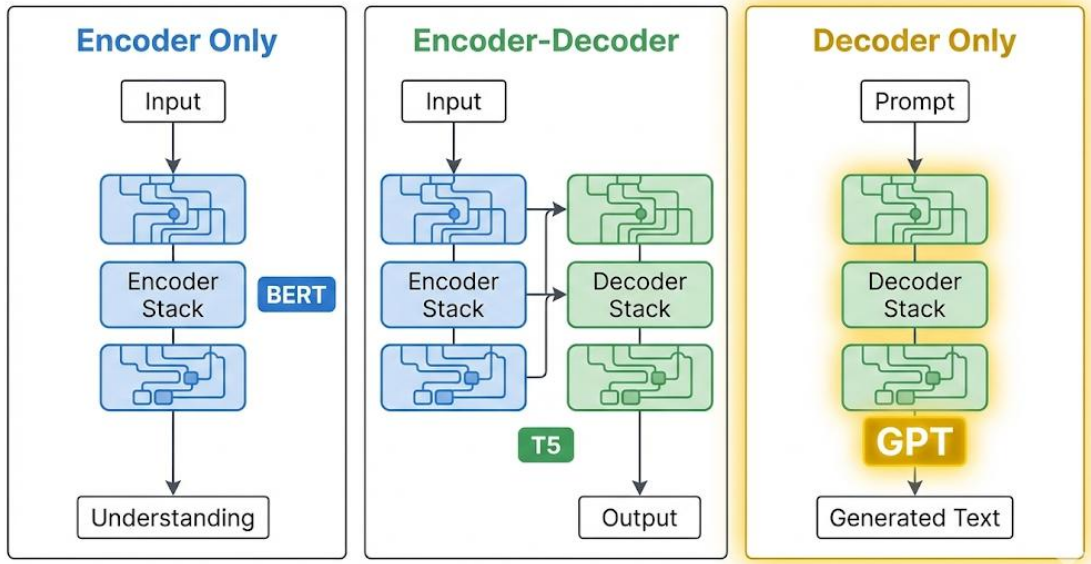
Three different architectures..



© 2025 TekFrameworks

Transformer Architectures — Who Does What?

Architecture	Example	What it's good at
Encoder Only	BERT	Understanding text (classification, search, Q&A)
Decoder Only	GPT	Generating text (chat, code, writing)
Encoder–Decoder	T5	Mapping input → output (translation, summarization)

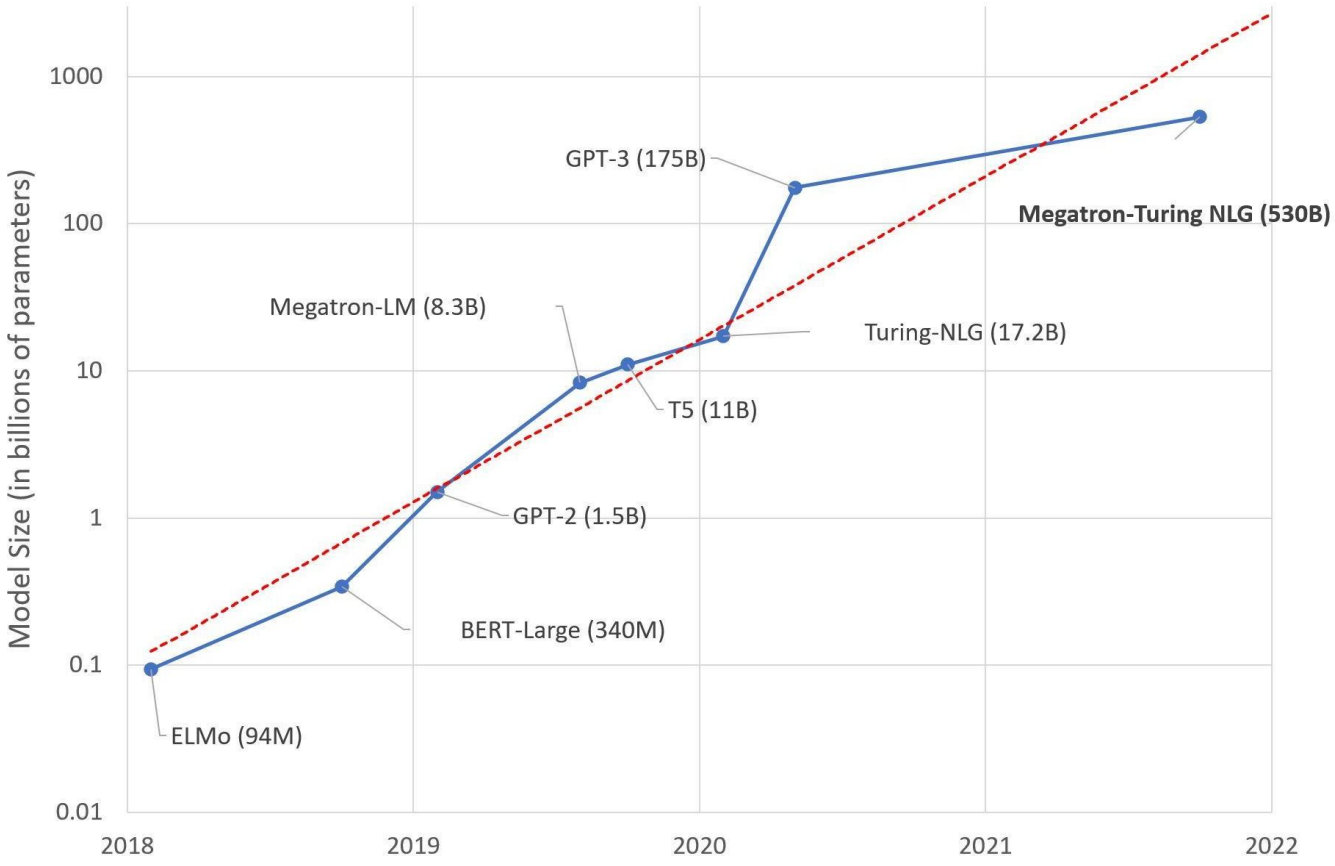


All modern LLMs use Transformers — but in different configurations depending on the task.

© 2025 TekFrameworks

What is a Large Language Model (LLM)?

- "Large" refers to the size of the models.
  - Size refers to the number of parameters of the model.
- LLM today refers to Transformer Architecture trained on unsupervised text data.



© 2025 TekFrameworks

Unsupervised LLM training

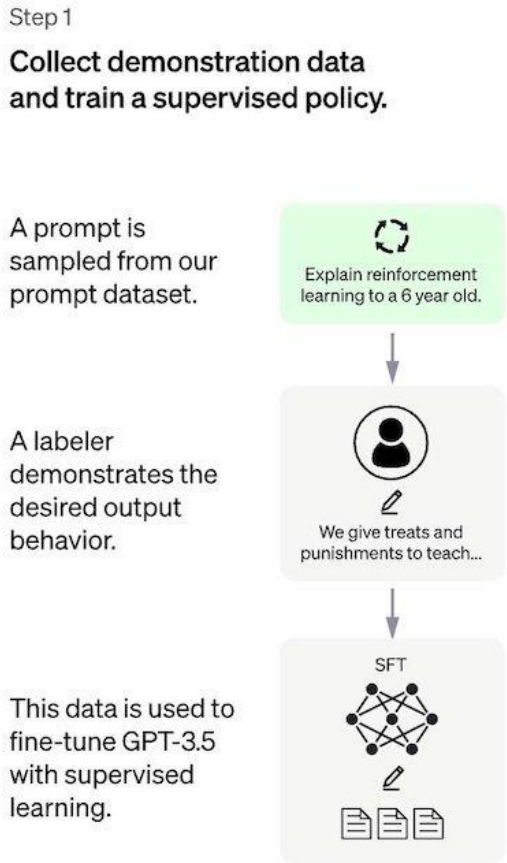
- The LLM after unsupervised training, understands the language.
- It can predict next word – but doesn't necessarily do the right thing.
- A prompt "What is the capital of India?" can potentially result in next sentence prediction of "What is the capital of USA?"

© 2025 TekFrameworks

Supervised Fine-Tuning

- A group of human labelers are asked to **write down the expected output for a list of prompts**. As this process is slow and expensive, the result is a relatively small, high-quality curated dataset (of approximately 12-15k data points, presumably) that is to be used to fine-tune a pre-trained language model.

- A GPT-3 model which was fine-tuned mostly on programming code was used as a baseline model. Therefore, in order to create a general-purpose chatbot like ChatGPT, the developers decided to **fine-tune on top of a "code model"** rather than a pure text model.



© 2025 TekFrameworks

SFT is not enough

WHY SUPERVISED FINE-TUNING ALONE IS INSUFFICIENT FOR ALIGNMENT

Grammatically Correct Responses vs. Human Preference

USER PROMPT

"How do I open a bank account?"

RESPONSE A (vague)
"Visit your nearest branch and ask for assistance."

RESPONSE B (overly technical)
"You must complete KYC documentation including Form 60, PAN verification, and AML compliance procedures."

RESPONSE C (unsafe / incomplete)
"Just give any ID and they will open it for you."

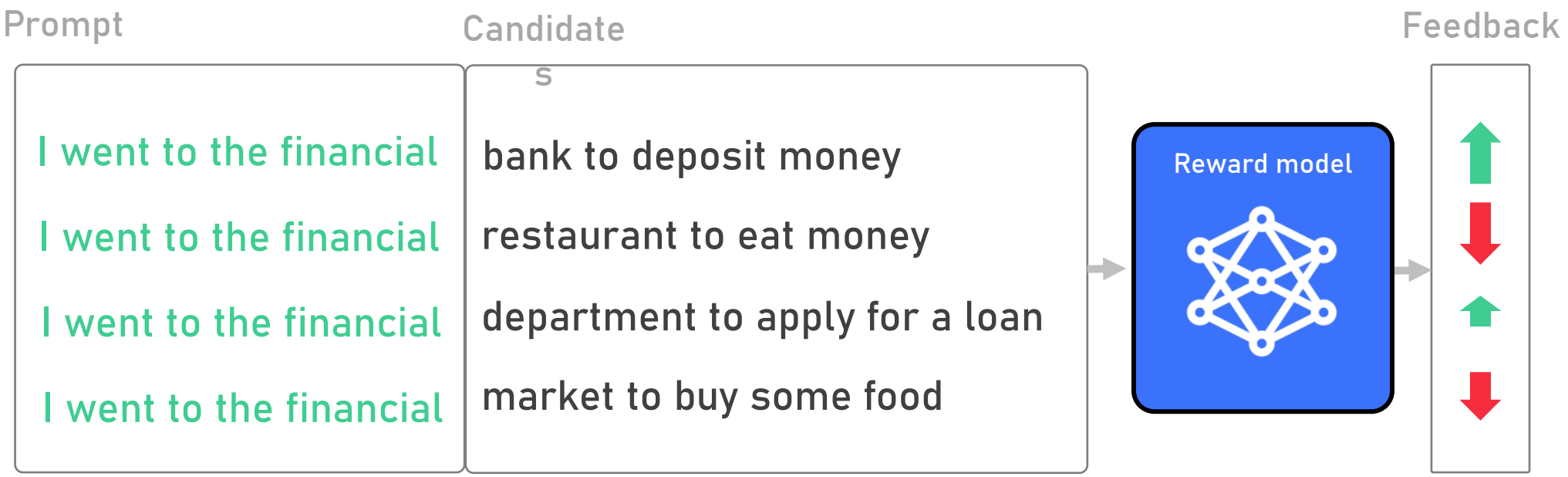
RESPONSE D (helpful)
"Choose a bank, bring valid ID and address proof, fill out the application form, and make an initial deposit."

All are grammatically correct. But which one do humans prefer?

Supervised fine-tuning teaches language — not preference.

© 2025 TekFrameworks

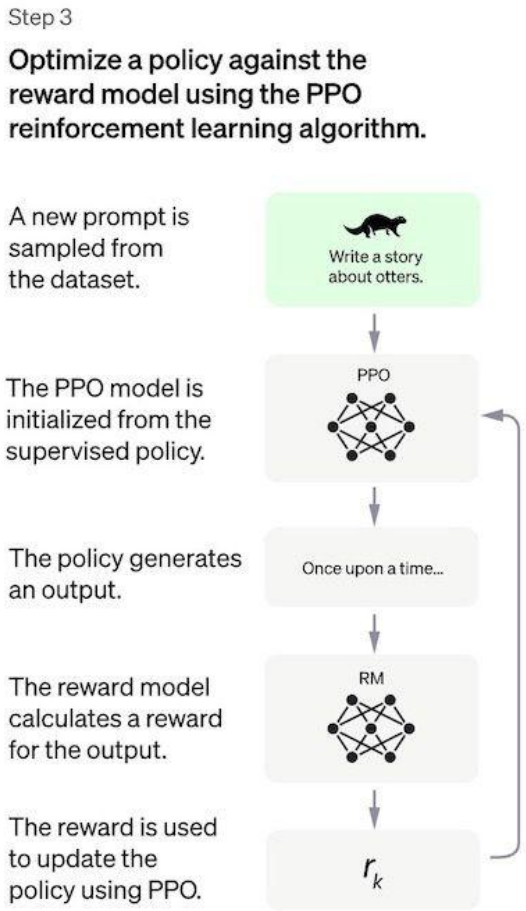
REINFORCEMENT LEARNING

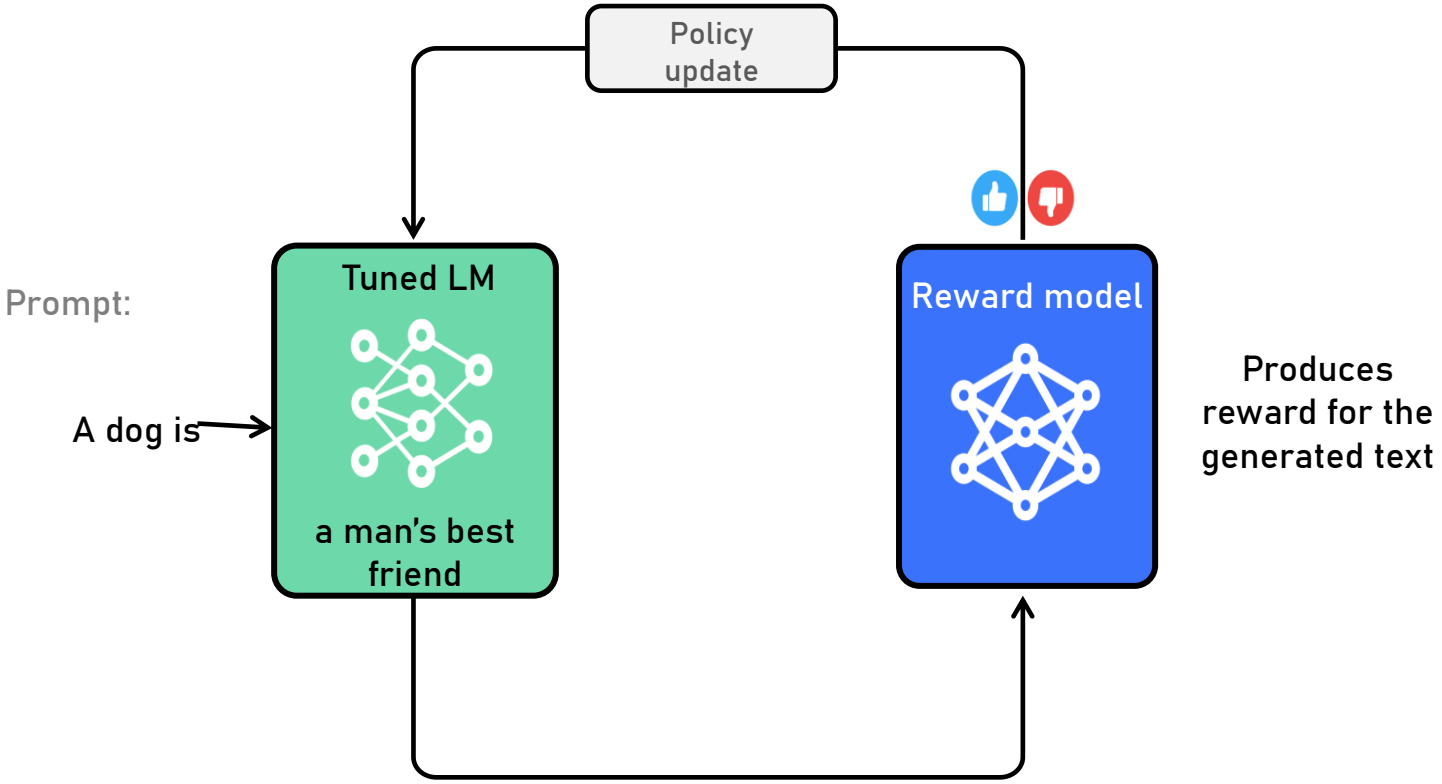


Another reward model is trained using the inputs(Prompt & candidates) & outputs(feedback)

© 2025 TekFrameworks

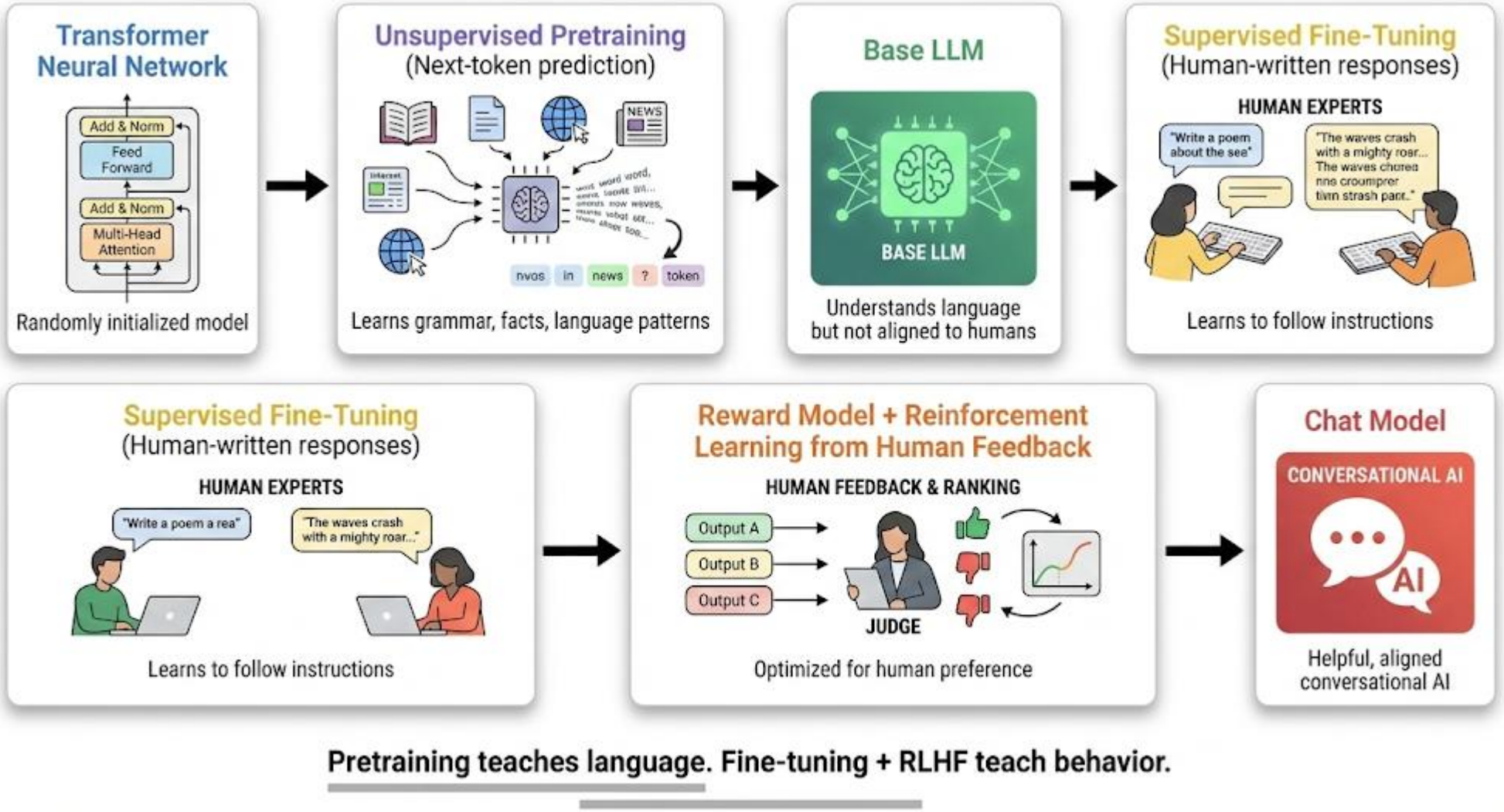
Reinforcement Learning





© 2025 TekFrameworks

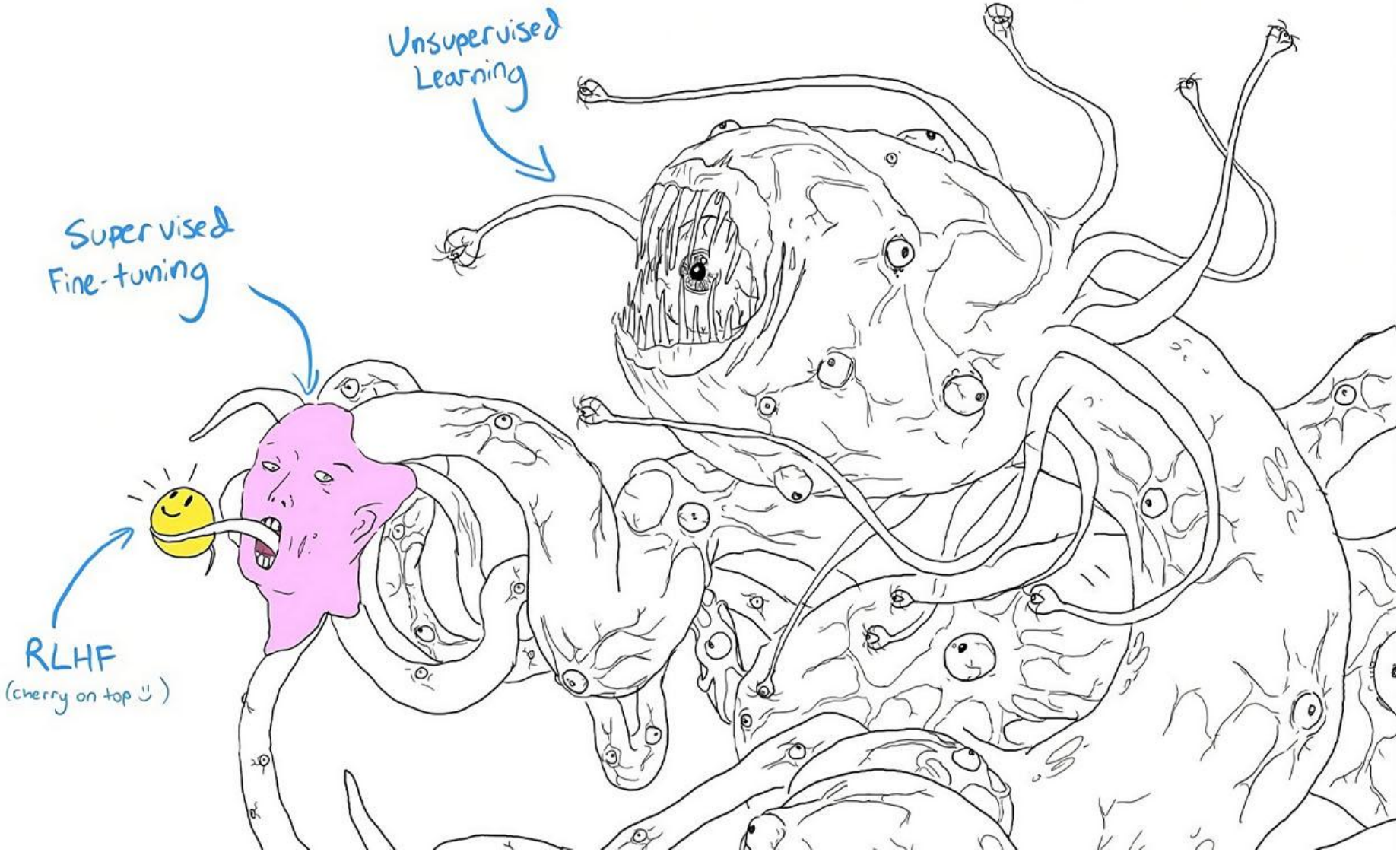
How a Transformer Becomes ChatGPT



Pretraining teaches language. Fine-tuning + RLHF teach behavior.

© 2025 TekFrameworks

How ChatGPT actually works..



© 2025 TekFrameworks

Current Landscape Mar 2026

LLM	Developer	Strengths	Key Use Cases	Type
GPT-5 (e.g., 5.2/5.4)	OpenAI	Multimodal, advanced reasoning (adaptive/deep thinking), large context (often 400k+), professional knowledge work	Content creation, code, multimodal tasks, complex reasoning, productivity	Proprietary
Claude (Opus/Sonnet 4.6)	Anthropic	Safety-first, very long context (up to 1M tokens beta), aligned output, excellent coding/agentic tasks, thoughtful reasoning	Document analysis, coding, agent workflows, customer support, complex long-running tasks	Proprietary
Gemini 3 (e.g., 3.1 Pro)	Google	Strong vision-language/multimodal, integrated with Google ecosystem/tools, very long context (up to 1M+), deep think modes	Multimodal tasks, productivity, search, research, visual explanations, complex problem-solving	Proprietary
Llama 4 (e.g., Scout/Maverick)	Meta AI	Open-weight, natively multimodal, MoE architecture, scalable, long context support, efficient on hardware	Research, local inference, fine-tuning, multimodal experiences, on-device/custom deployments	Open-weight
Mistral 3	Mistral AI	Small + efficient models, strong at code, fast inference, permissive licenses, good reasoning in newer versions	Real-time apps, IDE assistants, on-device AI, efficient enterprise use	Open-source/weight
DeepSeek V3.2	DeepSeek AI	High reasoning (often frontier-level), excellent for code/data tasks, low cost, MoE architecture, multilingual	Coding, RAG, data analysis, multilingual enterprise, agentic workflows	Open-weight, restricted license

© 2025 TekFrameworks

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks